

# COMPETITIVE PROGRAMMING

Complete Roadmap: Beginner → National Stage  
Java Edition • LeetCode + Codeforces + ICPC

## BRUTAL TRUTHS BEFORE YOU START

|                                      |                                                                            |
|--------------------------------------|----------------------------------------------------------------------------|
| ✗ Java is fine for interviews.       | At ICPC/CF Div.1+ level, C++ is the standard. Plan to switch eventually.   |
| ✗ No shortcuts exist.                | This is a 12–18 month commitment minimum to reach national level.          |
| ✓ LC easy ≠ CP ready.                | Codeforces problems are a different beast — math-heavy, tight constraints. |
| ✓ Consistency > intensity.           | 2 hours/day every day beats 10 hours on weekends.                          |
| ✓ Editorial reading is not cheating. | Reading after a genuine 45-min struggle is how you actually learn.         |

## THE 5 PHASES

| Phase               | Duration  | Target                                                    | CF Rating |
|---------------------|-----------|-----------------------------------------------------------|-----------|
| 1 — Foundation      | 6–8 wks   | Arrays, Two Pointers, Binary Search, Recursion, Sorting   | < 800     |
| 2 — Core DSA        | 8–10 wks  | Linked List, Stack, Queue, Trees, Graphs, DP Intro, Heaps | 800–1200  |
| 3 — Intermediate CP | 10–12 wks | Adv DP, Graph Algos, Segment Tree, Number Theory, Greedy  | 1200–1600 |
| 4 — Advanced CP     | 12–16 wks | Bitmask DP, Flows, Strings (KMP/Z/SA), Geometry, SCC      | 1600–2100 |
| 5 — National Level  | Ongoing   | ICPC, CodeChef Long, CF Div.1, Competitive Math           | 2100+     |

## PHASE 1 — FOUNDATION (Weeks 1–8)

### Arrays & Strings

- Kadane's Algorithm (Max Subarray)
- Prefix sums
- Frequency counting
- Two Sum variants

### Two Pointers

- Sorted array problems
- Container with most water
- 3Sum
- Trapping rain water

### Binary Search

- On sorted array
- On answer (not just index)

- Find first/last position
- Search in rotated array

## ■ Recursion

- Subsets
- Permutations
- Backtracking basics
- N-Queens

## ■ Sorting

- Merge sort (implement from scratch)
- Quick sort internals
- Custom comparators
- Counting sort

✓ **TIP: Solve 80–100 LC problems. All easy done, mediums flowing before Phase 2.**

## PHASE 2 — CORE DSA (Weeks 9–18)

### ■ Stack & Queue

- Monotonic stack (Next Greater Element)
- Min Stack
- Evaluate RPN
- Largest Rectangle in Histogram

### ■ Linked List

- Reverse, detect cycle, merge sorted
- LRU Cache
- Find middle node
- Reorder list

### ■ Trees

- DFS/BFS traversals
- LCA (Lowest Common Ancestor)
- Diameter, height, balance check
- Serialize/Deserialize

### ■ Graphs

- BFS/DFS on grid and adjacency list
- Dijkstra (shortest path)
- Union-Find (Disjoint Set)
- Topological sort

### ■ Dynamic Programming

- Fibonacci to memoization to tabulation
- 0/1 Knapsack
- Longest Common Subsequence
- Coin Change

### ■ Heap / Priority Queue

- Top K elements
- Merge K sorted lists

- Median of stream
- Task scheduler

✓ **TIP: Start Codeforces Div.3 contests. Target 150–250 LC problems total.**

## PHASE 3 — INTERMEDIATE CP (Weeks 19–30)

### ■ Advanced DP

- Bitmask DP
- Interval DP
- Tree DP
- Digit DP
- DP on Graphs

### ■ Advanced Graph

- Bellman-Ford
- Floyd-Warshall (all-pairs shortest path)
- SCC (Kosaraju/Tarjan)
- Bridges & Articulation Points

### ■ Segment Tree

- Point update, range query
- Range update with lazy propagation
- Merge sort tree

### ■ Number Theory

- Sieve of Eratosthenes
- GCD/LCM, modular arithmetic
- Modular inverse, fast exponentiation
- Euler's totient

### ■ Greedy

- Activity selection
- Fractional knapsack
- Huffman encoding
- Jump game variants

✓ **TIP: Participate in CF Div.2, CodeChef Starters. Target CF rating 1400+.**

## PHASE 4 — ADVANCED CP (Weeks 31–46)

### ■ String Algorithms

- KMP (pattern matching)
- Z-Algorithm
- Rabin-Karp hashing
- Suffix Array + LCP

### ■ Advanced Trees

- Fenwick Tree (BIT)
- Persistent Segment Tree
- Heavy-Light Decomposition

- Centroid Decomposition

## ■ Network Flow

- Max flow (Ford-Fulkerson)
- Min cut
- Bipartite matching

## ■ Geometry

- Convex hull
- Line intersection
- Point in polygon
- Rotating calipers

## ■ Competitive Math

- Combinatorics ( $nCr \bmod p$ )
- Matrix exponentiation
- Probability basics
- Game theory (Sprague-Grundy)

✓ **TIP: Solve CF 1800–2200 rated problems. Participate in CF Div.1+2 rounds.**

## ■ WEEKLY SCHEDULE

| Day                | Activity                                                                              | Time   |
|--------------------|---------------------------------------------------------------------------------------|--------|
| <b>Mon–Fri</b>     | 2 problems/day — timed 45 min each. One easy/medium + one medium/hard                 | 90 min |
| <b>Saturday</b>    | 1 full virtual contest — CF Div.3 or LC Weekly. No pausing, no hints                  | 2 hrs  |
| <b>Sunday</b>      | Review all unsolved. Read editorials. Recode from memory — never copy-paste           | 2 hrs  |
| <b>Every 2 wks</b> | Revisit problems you got wrong. If you can't solve in 20 min → you haven't learned it | 1 hr   |

## ■ PLATFORMS & WHEN TO USE THEM

| Platform                | When              | Focus                                                |
|-------------------------|-------------------|------------------------------------------------------|
| <b>LeetCode</b>         | Phase 1–2 primary | Interview prep, DSA foundations                      |
| <b>Codeforces</b>       | Phase 2 onwards   | CP contests, rating, Div.3→Div.1                     |
| <b>CSES Problem Set</b> | Phase 2–3         | Best structured CP problem set. Do all of it.        |
| <b>CodeChef</b>         | Phase 3 onwards   | Long challenges, Indian CP community, ICPC qualifier |
| <b>AtCoder</b>          | Phase 3 onwards   | Elegant problems, great for math/DP                  |
| <b>ICPC</b>             | Phase 4–5         | National target — form a 3-person team               |

## ■ NON-NEGOTIABLES

1. Never look at a solution before 45 minutes of genuine struggle.
2. After reading an editorial — close it. Code the solution from memory.
3. Track every problem: topic, difficulty, attempts, pattern recognized.

4. Never use static arrays for memoization — initialize inside the function.
5. Always use ArrayDeque, never the Stack class.
6. Compare Integer objects with .equals(), never ==.
7. Store indices in monotonic stack, not values — unless values are sufficient.
8. Brute force first, always. Optimize only after you understand the problem.
9. One contest per week minimum — no excuses. Virtual counts.
10. After 3 wrong attempts on a contest problem → read editorial, no shame.

## PATTERN RECOGNITION CHEAT SHEET

| Signal in problem                          | Pattern to use                |
|--------------------------------------------|-------------------------------|
| "Next greater/smaller element"             | Monotonic Stack               |
| "Subarray with condition" / sliding window | Two Pointers / Sliding Window |
| "K smallest/largest"                       | Min/Max Heap                  |
| "Shortest path in unweighted graph"        | BFS                           |
| "Shortest path with weights"               | Dijkstra                      |
| "Number of ways to reach..."               | DP (usually)                  |
| "All subsets / permutations"               | Backtracking                  |
| "Range sum query"                          | Prefix Sum or Segment Tree    |
| "Connected components"                     | Union-Find or DFS             |
| "Sorted + find target"                     | Binary Search                 |
| "Interval overlapping / merging"           | Sort by start, greedy         |
| "Anagram / character frequency"            | HashMap with freq count       |
| "Top K frequent"                           | Heap + HashMap                |
| "LRU Cache"                                | HashMap + Doubly Linked List  |

## JAVA QUICK REFERENCE

| Structure       | Declaration                                                           | Key Methods                                    |
|-----------------|-----------------------------------------------------------------------|------------------------------------------------|
| <b>Stack</b>    | Deque s = new ArrayDeque<>()                                          | push(), pop(), peek(), isEmpty()               |
| <b>Queue</b>    | Queue q = new ArrayDeque<>()                                          | offer(), poll(), peek(), isEmpty()             |
| <b>Min Heap</b> | PriorityQueue pq = new PriorityQueue<>()                              | offer(), poll(), peek()                        |
| <b>Max Heap</b> | PriorityQueue pq = new<br>PriorityQueue<>(Collections.reverseOrder()) | offer(), poll(), peek()                        |
| <b>HashMap</b>  | Map map = new HashMap<>()                                             | put(), get(), containsKey(), computeIfAbsent() |
| <b>HashSet</b>  | Set set = new HashSet<>()                                             | add(), contains(), remove()                    |

## ■ MILESTONES TO TRACK PROGRESS

| ✓ | Timeline | Milestone                                              |
|---|----------|--------------------------------------------------------|
| ■ | Month 1  | 50 LC problems solved, all easy done                   |
| ■ | Month 2  | 100 LC problems, first CF contest participated         |
| ■ | Month 3  | 150 LC, CF rating 800+, first Div.3 solved 3 problems  |
| ■ | Month 4  | 200 LC, CF rating 1000+, CSES 30 problems done         |
| ■ | Month 6  | 250 LC, CF rating 1200+, comfortable with DP basics    |
| ■ | Month 9  | 350 LC, CF rating 1500+, CSES 100+ problems done       |
| ■ | Month 12 | CF rating 1800+, Segment Tree + Adv DP solid           |
| ■ | Month 18 | CF rating 2100+, ICPC team formed, regional qualifiers |
| ■ | National | CF 2200+, consistent Div.1 performance, ICPC regionals |

---

The gap between you and a national-level coder is not talent — it's solved problems and contests participated in.